



Marco Faella

Classi interne, locali ed anonime

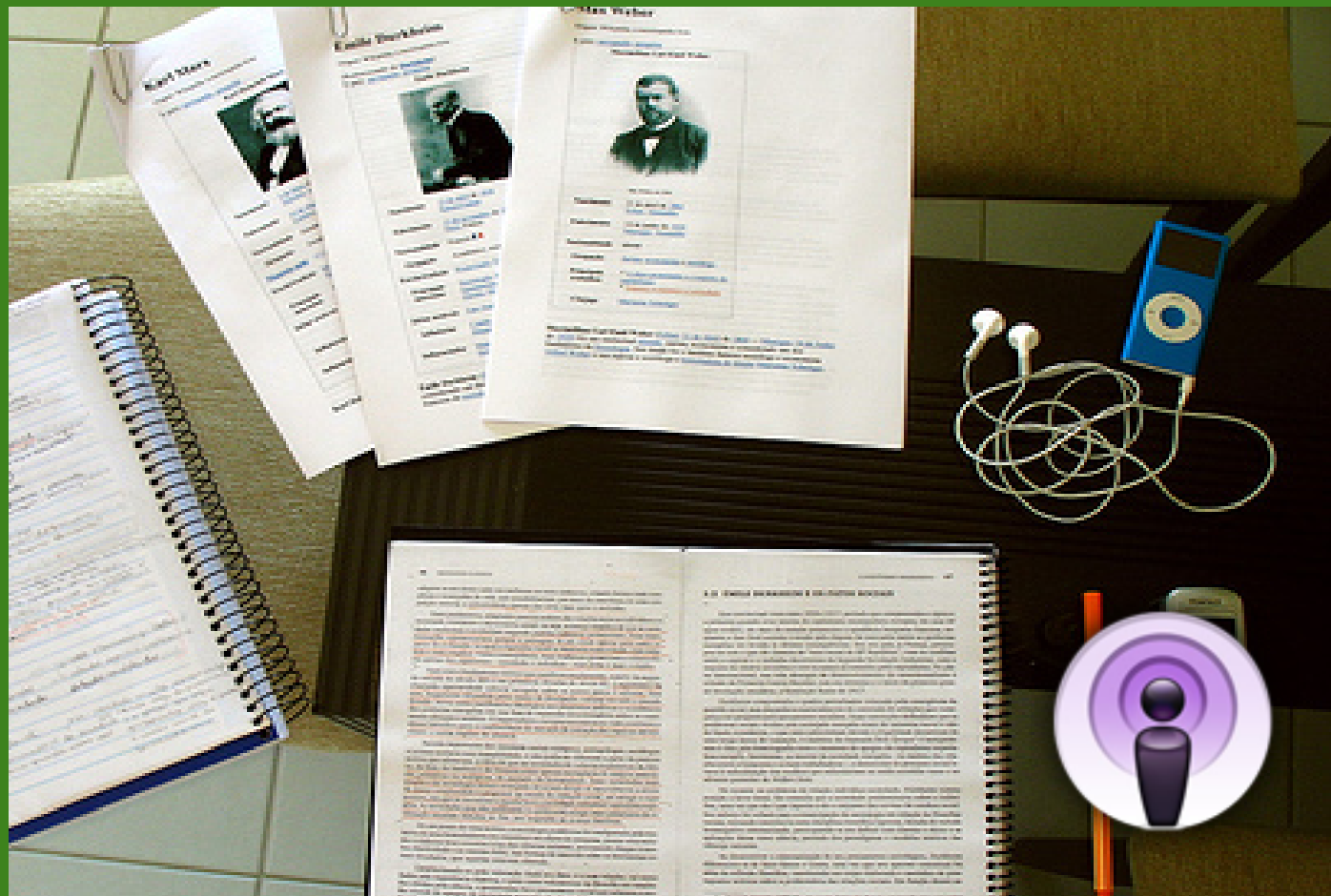
Lezione n. 6
Parole chiave:
Java

Corso di Laurea:
Informatica

Insegnamento:
Linguaggi di Programmazione II

Email Docente:
faella.didattica@gmail.com

A.A. 2009-2010



- Java permette di definire una classe (o interfaccia) all'interno di un'altra
- Queste classi vengono chiamate "interne" o "annidate"
 - in inglese, il termine usato è *nested*
 - In particolare, le classi interne non statiche sono chiamate *inner*
- Tale meccanismo arricchisce le possibilità di relazioni tra classi, introducendo in particolare **nuove regole di visibilità**
- Se la definizione di una classe si trova all'interno di un metodo, tale classe viene chiamata **locale**
- Infine, una classe locale può anche essere **anonima**, quando il suo nome non sarebbe rilevante e/o utile

Le classi interne (non statiche) godono delle seguenti proprietà distintive:

1) Privilegi di visibilità rispetto alla classe contenitrice e alle altre classi in essa contenute

- permettono una stretta collaborazione tra queste classi

2) Restrizioni di visibilità rispetto alle classi esterne a quella contenitrice

- permettono di nascondere la classe all'esterno (incapsulamento)

3) Un riferimento implicito ad un oggetto della classe contenitrice

- ogni oggetto della classe interna “conosce” l'oggetto della classe contenitrice che l'ha creato

- Una classe interna dichiarata **all'interno di un metodo** viene chiamata classe *locale*
- Una classe locale **non** ha modificatore di visibilità, in quanto è visibile **solo all'interno del metodo** in cui è dichiarata
- Una classe locale **non** può avere il modificatore *static*, in quanto eredita il suo essere statica o meno dal metodo in cui è dichiarata
- Oltre a godere delle proprietà comuni alle classi interne, le classi locali “vedono” le variabili locali e i parametri formali del metodo in cui sono contenute, a patto che essi siano *effectively final*
- *Effectively final* = usato come se fosse final
- La slide successiva illustra questa, apparentemente arbitraria, restrizione

- Le istanze di una classe locale possono **vivere più a lungo** del metodo in cui la loro classe è visibile
- Tipicamente, questo succede quando un oggetto di una classe locale viene restituito dal metodo, mascherato da una superclasse o super-interfaccia nota all'esterno
- In questi casi, l'accesso alle variabili locali (compresi i parametri formali) di quel metodo può avvenire quando quel metodo è ormai terminato, cancellando di fatto quelle variabili
 - si ricordi che le variabili locali e i parametri formali sono allocati sullo stack e quindi **vengono cancellati al termine di ciascuna invocazione** del metodo
- Pertanto, per dare l'illusione al programmatore di accedere a quelle variabili, il compilatore in realtà inserisce negli oggetti delle classi locali delle *copie* di quelle variabili
- Se il meccanismo funzionasse anche per variabili non final, il “trucco” delle copie delle variabili locali diventerebbe visibile al programmatore, invece che nascosto
 - Infatti, potrebbe accadere che una variabile locale venisse modificata dopo la creazione di un oggetto della classe interna
 - L'oggetto della classe interna, interrogato successivamente, ricorderebbe il vecchio valore di quella variabile, invece del valore modificato

Nella libreria AWT (Abstract Windowing Toolkit) per le interfacce grafiche, l'interfaccia ActionListener rappresenta un oggetto in grado di rispondere ad un evento

```
interface ActionListener {  
    void actionPerformed(ActionEvent e);  
}
```

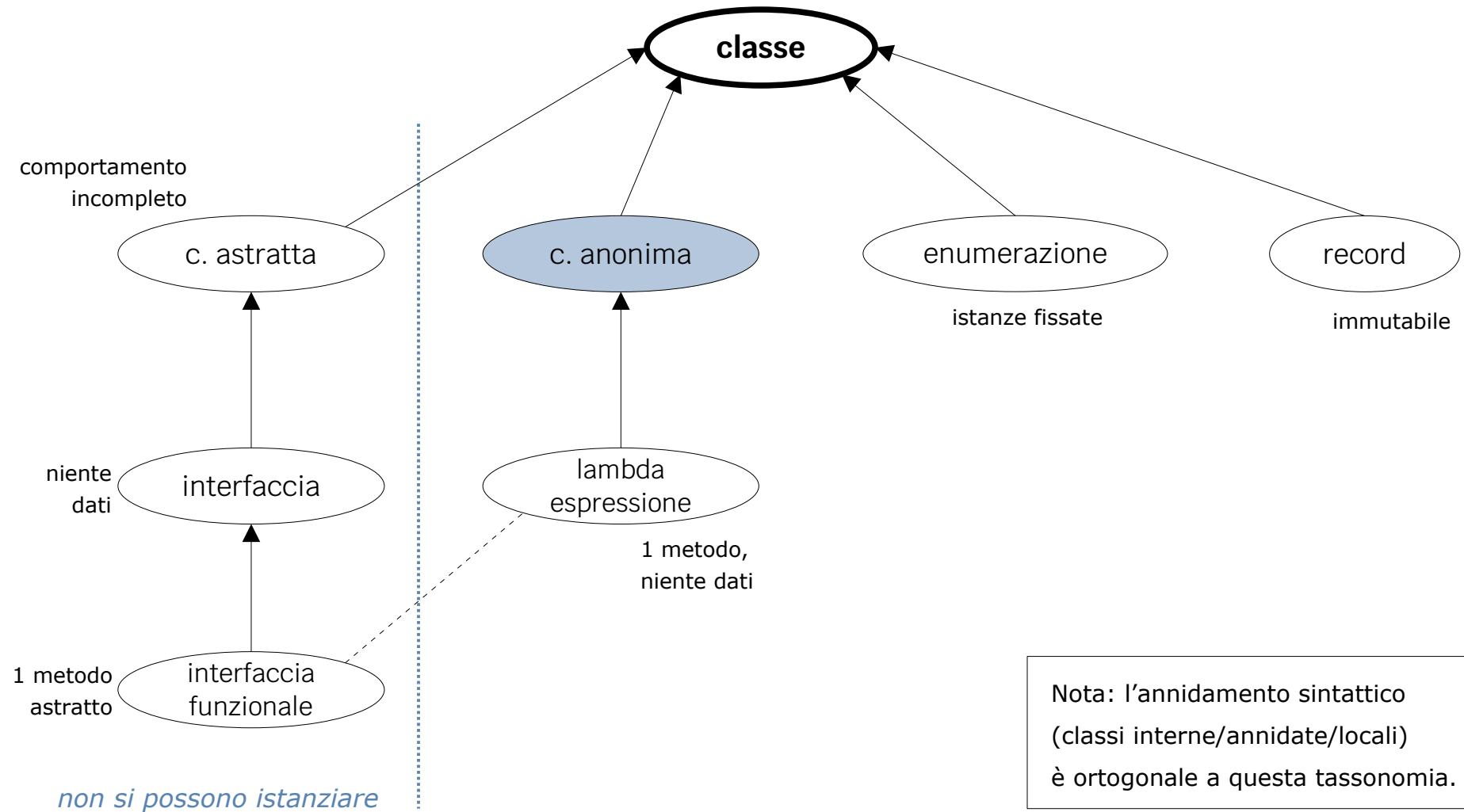
- Il seguente frammento crea due widget grafici:
 - una scritta statica (cioè non editabile)
 - un pulsante

```
final JLabel label = new JLabel("Hello");  
JButton b = new JButton(...);
```

- Alla pressione del pulsante, vogliamo modificare il contenuto della scritta
- Usiamo una classe locale per creare un oggetto che implementi ActionListener

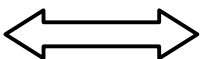
```
class MyActionListener implements ActionListener {  
    public void actionPerformed(ActionEvent e) {  
        // reagisci alla pressione del tasto b  
        label.setText("Goodbye");  
    }  
}  
  
b.addActionListener(new MyActionListener());
```

Classi anonime



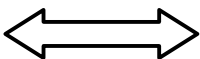
- Talvolta una classe interna viene utilizzata **soltanto una volta**, tipicamente per istanziare un oggetto che poi viene mascherato con una classe o interfaccia nota
- In questi casi, si può utilizzare una classe anonima
- Una classe anonima si definisce *a partire da una classe o un'interfaccia esistente*
- Nella seguente tabella, a sinistra riportiamo la sintassi per la creazione di classi anonime a partire da una classe o interfaccia esistente
- A destra riportiamo il costrutto equivalente utilizzando normali classi con nome

```
new Classe(exp_1, ..., exp_n) {
    <body>
}
```



```
class X extends Classe {
    public X(T_1 x_1, ..., T_n x_n) {
        super(x_1, ..., x_n);
    }
    <body>
}
new X(exp_1, ..., exp_n)
```

```
new Interfaccia() {
    <body>
}
```



```
class X implements Interfaccia {
    <body>
}
new X()
```

- Nel seguente frammento ispirato alla libreria di GUI Swing/AWT viene usata una classe anonima per creare un oggetto che implementi ActionListener

```
JButton b = new JButton(...);  
  
b.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        // reagisci alla pressione del tasto b  
    }  
} );
```

- Consideriamo la solita classe `Employee`, dotata di campi `nome` e `salario`
- Il seguente metodo usa una classe anonima per creare un `Employee` dotato di un titolo (Dott., Prof., etc.)
- Il titolo comparirà nella stringa restituita dal metodo `toString`

```
public static Employee makeSpecialEmployee(String myTitle, String myName, int mySalary)
{
    return new Employee(myName, mySalary) {
        @Override
        public String toString() {
            return myTitle + " " + getName() + ": " + getSalary();
        }
    };
}
```

Il parametro formale `"myTitle"` è visibile all'interno della classe anonima in quanto *effectively final*

- La figura rappresenta il memory layout delle classi definite di seguito
- Notate in particolare la differenza tra classe interna e sottoclasse

```
class A {  
    private int x;  
    private class B {  
        private int y;  
    }  
    public static class C extends A {  
        private int z;  
    }  
    public Object f(final int n) {  
        class Locale {  
            private int m = n;  
        }  
        return new Locale();  
    }  
    public static void main(String[] args) {  
        A a = new A();  
        B b = a.new B();  
        C c = new C();  
        Object o = a.f(7);  
    }  
}
```

- La figura rappresenta il memory layout delle classi definite di seguito
- Notate in particolare la differenza tra classe interna e sottoclasse

```
class A {
    private int x;
    private class B {
        private int y;
    }
    public static class C extends A {
        private int z;
    }
    public Object f(final int n) {
        class Locale {
            private int m = n;
        }
        return new Locale();
    }
    public static void main(String[] args) {
        A a = new A();
        B b = a.new B();
        C c = new C();
        Object o = a.f(7);
    }
}
```

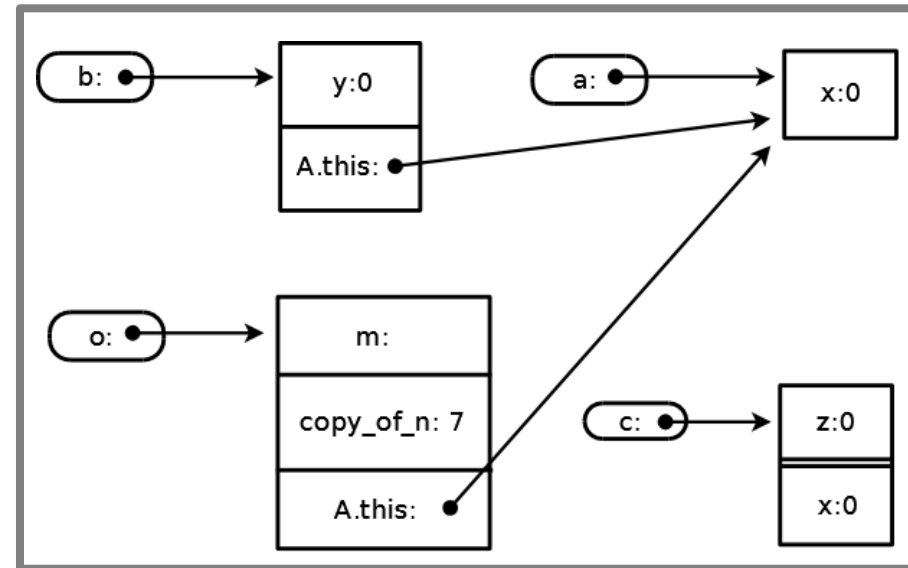


Figura 1: Il memory layout del frammento di programma a sinistra. In particolare, sono mostrati i quattro oggetti creati dal main.

- **C++**
 - [*Come Java*] Prevede classi interne (nested) con restrizioni di visibilità arbitraria (ad es., *private*)
 - [*Come Java*] La classe interna ha accesso a tutti i membri della contenitrice
 - [*Diverso*] La classe contenitrice *non* ha accesso privilegiato al contenuto di quella interna
 - [*Diverso*] Non prevede riferimento implicito alla classe contenitrice
 - Privilegi di visibilità si ottengono anche tramite il costrutto *friend*
- **C#**
 - Regole analoghe a C++
 - Nessun costrutto *friend*
 - Le linee guida MS sconsigliano esplicitamente di creare classi interne pubbliche

Nell'ambito di un programma di geometria, si implementi la classe Triangolo, il cui costruttore accetta le misure dei tre lati. Se tali misure non danno luogo ad un triangolo, il costruttore deve lanciare un'eccezione. Il metodo getArea restituisce l'area di questo triangolo.

Si implementino anche la classe Triangolo.Rettangolo, il cui costruttore accetta le misure dei due cateti, e la classe Triangolo.Isoscele, il cui costruttore accetta le misure della base e di uno degli altri lati.

Si ricordi che:

- Tre numeri a , b e c possono essere i lati di un triangolo a patto che $a < b + c$, $b < a + c$ e $c < a + b$.
- L'area di un triangolo di lati a , b e c è data da: $\frac{p(p-a)(p-b)(p-c)}{4}$ (formula di Erone), dove p è il semiperimetro.

Esempio d'uso (fuori dalla classe Triangolo):

```
Triangolo x = new Triangolo(10,20,25);  
Triangolo y = new Triangolo.Rettangolo(5,8);  
Triangolo z = new Triangolo.Isoscele(6,5);
```

```
System.out.println(x.getArea());  
System.out.println(y.getArea());  
System.out.println(z.getArea());
```

Output dell'esempio d'uso:

```
94.9918  
19.9999  
12.0
```